ML4N - SSH Shell Attack Analysis

SATTAR AFTABI AMANEH, Politecnico di Torino, Italy
DONYA MOHEBI, Politecnico di Torino, Italy
MICHELE VALERIO, Politecnico di Torino, Italy
THIAGO RICARDO, Politecnico di Torino, Italy

This paper presents a comprehensive machine learning approach to analyze SSH shell attack sessions collected from honeypot systems. Using a dataset of 233,035 unique Unix shell attack sessions spanning from June 2019 to February 2020, we developed a multi-faceted methodology combining supervised learning, unsupervised clustering, and advanced language models to classify attacker tactics based on the MITRE ATT&CK framework.

The key contributions of this work include: (1) development of a robust preprocessing pipeline analyzing temporal trends, extracting numerical features, and evaluating intent distributions from large-scale SSH attack session data; (2) implementation of supervised classification models (Logistic Regression and SVM) with hyperparameter tuning achieving micro F1-scores above 0.99 and macro F1-scores of 0.846 for multi-label intent prediction; (3) application of unsupervised clustering techniques (K-Means with k=10 and Agglomerative Hierarchical Clustering) to uncover hidden patterns in attack behaviors with cluster purity scores of 97.75%; and (4) comparative evaluation of BERT-based language models against traditional ML approaches for intent classification.

Our analysis reveals that Discovery and Persistence are the dominant attack intents, often occurring together, while extreme class imbalance remains the primary challenge limiting performance on rare intents. Traditional machine learning with carefully tuned Logistic Regression achieves superior overall performance compared to BERT, demonstrating that proper feature engineering and class weighting are more effective than architectural sophistication when discriminative lexical features exist.

CCS Concepts: • **Security and privacy → Intrusion detection systems**; • **Computing methodologies → Machine learning**; *Neural networks*; *Unsupervised learning*.

Additional Key Words and Phrases: Machine learning, supervised learning, unsupervised learning, language models, text classification, clustering, intent classification, SSH shell attacks, security log analysis, MITRE ATT&CK

1 Introduction

1.1 Motivation

Security logs play a crucial role in understanding and mitigating cyber attacks, particularly in the domain of network and system security. With the increasing sophistication of cyber threats, analyzing and interpreting security logs has become paramount for detecting, preventing, and responding to potential security breaches. Unix shell attacks, especially those executed through the SSH protocol, represent a significant vector for potential system compromises.

The complexity of security log analysis stems from several key challenges:

- Logs are often unstructured and contain ambiguous or malformed text
- Manual parsing and interpretation of logs is time-consuming and error-prone
- The sheer volume of log data makes comprehensive manual review impractical
- Multi-label nature of attacks where single sessions exhibit multiple tactics simultaneously

These challenges underscore the need for automated, intelligent approaches to log analysis that can efficiently extract meaningful insights and identify potential security threats.

Authors' Contact Information: Sattar Aftabi Amaneh, Politecnico di Torino, Turin, Italy; Donya Mohebi, Politecnico di Torino, Turin, Italy; Michele Valerio, Politecnico di Torino, Turin, Italy; Thiago Ricardo, Politecnico di Torino, Turin, Italy.

1.2 Objective

The primary objective of this research is to develop and evaluate machine learning techniques for automatic analysis and classification of SSH shell attack logs. Specifically, we aim to:

- Automate multi-label intent classification using MITRE ATT&CK framework
- Compare traditional machine learning approaches with modern language models to identify optimal solutions for production deployment
- Discover latent attack patterns through unsupervised clustering to reveal attacker strategies beyond labeled intents
- Provide security professionals with actionable insights into attack behaviors, temporal patterns, and threat prioritization
- Evaluate the impact of extreme class imbalance on model performance and identify effective mitigation strategies

The significance of this research lies in its potential to enhance cybersecurity threat detection and response capabilities by transforming complex, unstructured log data into actionable intelligence. By systematically comparing supervised classification, unsupervised pattern discovery, and advanced language models, this work provides practical guidance for deploying ML-powered log analysis systems in real-world security operations.

1.3 MITRE ATT&CK Framework

The MITRE ATT&CK (Adversarial Tactics, Techniques, and Common Knowledge) framework provides a comprehensive knowledge base of adversary tactics and techniques observed in real-world cyber attacks. This framework serves as a standardized methodology for understanding and categorizing attack strategies.

For our research, we focus on seven key intents derived from the MITRE ATT&CK framework:

- **Persistence**: Methods adversaries use to maintain access to systems after restarts or credential changes
- **Discovery**: Techniques for gathering information about the target system and network environment
- **Defense Evasion**: Strategies to avoid detection by security mechanisms
- **Execution**: Techniques for running malicious code on target systems
- **Impact**: Actions aimed at manipulating, interrupting, or destroying systems and data
- **Other**: Less common tactics including Reconnaissance, Resource Development, Initial Access, etc.
- **Harmless**: Non-malicious code or actions

1.4 Research Approach

Our research employs a multi-faceted approach combining four complementary methodologies:

(1) **Data Exploration and Preprocessing**:
   - Analysis of 233,035 SSH attack sessions with temporal trend identification
   - Feature extraction using TF-IDF vectorization for command-based representations
   - Investigation of extreme class imbalance across seven MITRE ATT&CK intent categories

(2) **Supervised Learning for Intent Classification**:
   - Implementation and comparison of traditional ML models (e.g., tuned Logistic Regression)
   - Comprehensive hyperparameter tuning with cross-validation
   - Evaluation of class weighting strategies to address minority class challenges

(3) **Unsupervised Clustering for Pattern Discovery**:
   - Application of K-Means and Agglomerative Hierarchical Clustering
   - Optimal cluster selection using multiple evaluation metrics
   - Analysis of cluster-intent relationships to validate discovered patterns

(4) **Language Model Exploration**:
- Fine-tuning BERT for multi-label SSH attack classification
- Comparative evaluation against traditional ML to assess transfer learning benefits, despite computational trade-offs
- Analysis of efficiency metrics (training time, inference latency, model size)

This comprehensive methodology enables us to identify the most effective approaches for SSH attack analysis while providing insights into the strengths and limitations of each technique in the context of security log analysis.

1.5 Related Work

Security log analysis has evolved significantly with advances in machine learning. Honeypot systems like those described by Provos [6] have become essential for collecting real-world attack data, enabling data-driven security research. Traditional signature-based intrusion detection systems struggle with novel attacks, motivating the shift toward behavioral analysis using machine learning [7].

Recent work in log analysis has demonstrated the effectiveness of deep learning approaches. Du et al. [8] applied LSTMs to system logs for anomaly detection, while transformer-based models like BERT have shown promise in understanding command sequences and security contexts. Multi-label classification in security domains presents unique challenges due to class imbalance and overlapping attack tactics [3].

Our work extends this body of research by combining traditional ML, unsupervised clustering, and BERT-based language models specifically for SSH attack analysis mapped to the MITRE ATT&CK framework. This multi-faceted approach enables both automated classification and discovery of novel attack patterns, addressing gaps in purely supervised or unsupervised methodologies. Building on this foundation, the following sections detail our methodology, experimental results, and key findings.

2 Data Exploration and Preprocessing

2.1 Dataset Overview

The dataset consists of SSH attack logs from honeypot systems, with each entry including:
- `session_id`: Unique integer identifier
- `full_session`: Command sequence (string)
- `first_timestamp`: Session start (datetime)
- `Set_Fingerprint`: MITRE ATT&CK labels (multi-label)

It contains **233,035 unique sessions** from June 2019 to February 2020, offering a rich view of SSH attack patterns.

2.2 Dataset Preparation and Feature Extraction

The Parquet file was preprocessed as follows:

(1) Decode `full_session` to readable format
(2) Convert `first_timestamp` to datetime
(3) Split sessions using ;, |, and whitespace
(4) Clean commands with regex to remove symbols/assignments
(5) Check for missing values (none found)

Example raw:

```
enable ; system ; shell ; sh ; cat /proc/mounts ;
/bin/busybox SAEMW ; cd /dev/shm ; cat .s || cp /bin/echo .s
```

Processed:

```
[enable, system, shell, sh, cat, /proc/mounts, /bin/busybox,
SAEMW, cd, /dev/shm, cat, .s, cp, /bin/echo, .s, ...]
```

2.3 Temporal Analysis

Attacks show continuous activity with notable patterns:

- Surge mid-October 2019 to January 2020, peaking December (~78,000)
- Uniform across days, slightly higher weekdays (Monday max ~35,000)
- Evening concentration (~21:00 UTC peak ~10,000), suggesting automated bots

Figure 1 captures these trends, highlighting 24/7 operations.

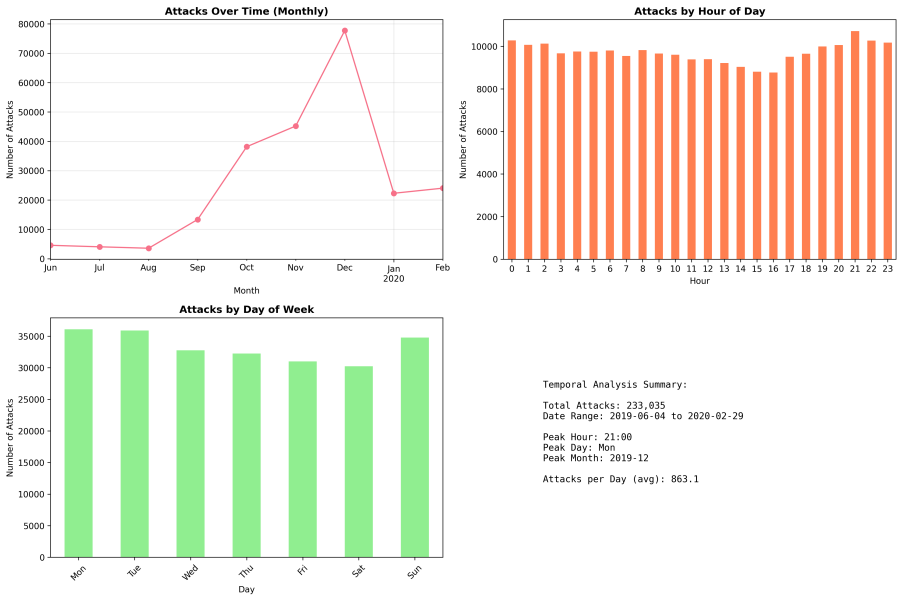

Fig. 1. Temporal attack distribution: Monthly peaks December 2019; hourly evening bias; weekday slight increase.

2.4 Session Characteristics Analysis

Sessions vary widely: median 116 words (mean 121) and 620 characters (mean 1,014), with complex attacks reaching 35,000+ words and 200,000+ characters due to long-tail distribution. This diversity reflects simple probes to multi-stage scripts. See Appendix A.1 for distributions.

2.5 Vocabulary Analysis

Recon commands dominate (grep, cat, echo, uname), with paths like /proc/cpuinfo for fingerprinting. TF-IDF representation effectively captured discriminative patterns, reducing dimensionality from 300,000 unique tokens to 5,000 most informative features (min_df=5, max_features=5000). See Appendix A.2 for top tokens.

2.6 Intent Distribution Analysis

Figure 2 shows most sessions have 2-3 intents (98.3%), averaging 2.39. Figure 3 highlights imbalance: Discovery (232,145, 99.6%), Persistence (211,295, 90.7%), Execution (92,927, 39.9%), Defense Evasion (18,999, 8.2%), Harmless (2,206, 0.95%), Other (327, 0.14%), Impact (27, 0.01%). Discovery/Persistence often co-occur.

Temporal evolution (Appendix A.3) shows stable majors, with Execution/Impact spiking December 2019.

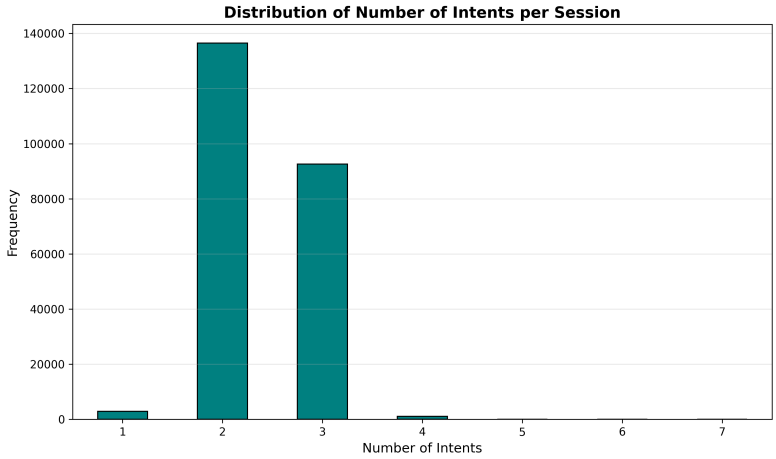

Fig. 2. Multi-label distribution: 58.6% 2 intents, 39.7% 3, rare singles (1.3%) or 4+ (0.5%).

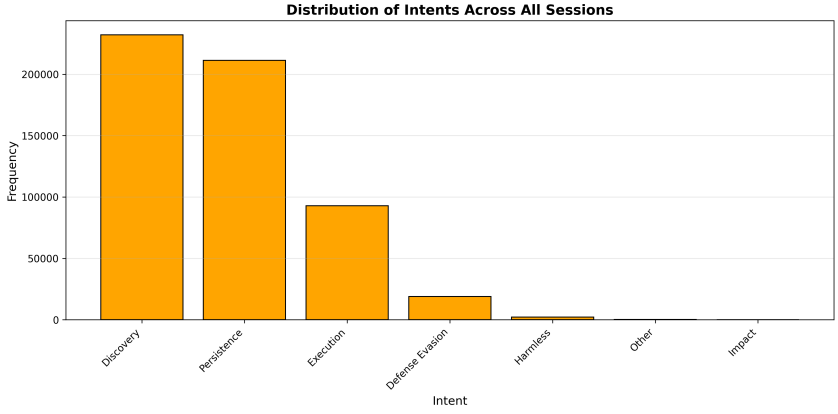

Fig. 3. Intent occurrence frequencies showing severe class imbalance. Discovery (99.6%) and Persistence (90.7%) dominate, while Impact (0.01%) is extremely rare.

2.7 Text Representation

Bag of Words (BoW) represents sessions as frequency vectors but ignores term importance. TF-IDF extends BoW by weighting terms based on inverse document frequency:

$$\text{TF-IDF}(t, d, D) = \text{TF}(t, d) \times \log\left(\frac{|D|}{|\{d : t \in d\}|}\right) \tag{1}$$

TF-IDF representation effectively captured discriminative attack patterns, reducing dimensionality from approximately 300,000 unique tokens to 5,000 most informative features through filtering (min_df=5) and selection (max_features=5000). This TF-IDF representation with 5,000 features was used for all subsequent supervised classification and unsupervised clustering tasks, providing an optimal balance between feature expressiveness and computational tractability.

3 Supervised Learning - Classification

3.1 Problem Formulation

This task is multi-label classification of sessions into MITRE ATT&CK intents, where each can have multiple labels:

- **Strategy**: Binary Relevance for independent predictions
- **Features**: TF-IDF vectors (5,000 dimensions)
- **Split**: 80% train (186,428 sessions), 20% test (46,607)
- **Metrics**: Precision, Recall, F1 (Micro/Macro), Hamming Loss

3.2 Models and Methodology

We evaluated two models for multi-label classification using One-vs-Rest strategy:

Logistic Regression. as baseline linear classifier:

- One-vs-Rest for multi-label
- L2 regularization (C=1.0 default)
- max_iter=1000 for convergence
- lbfgs solver for efficiency
- random_state=42 for reproducibility

Linear SVM. for efficient linear classification on high-dimensional text features::

- One-vs-Rest for multi-label
- Linear kernel (suitable for high-dimensional TF-IDF features)
- Regularization parameter C=1.0
- Balanced class weights to handle imbalance
- random_state=42 for reproducibility

Hyperparameter Tuning. via GridSearchCV (5-fold CV, scoring='f1_macro'):

- LR: C ∈ {0.1, 1, 10}, penalty ∈ {'l1', 'l2'} (with solver='liblinear' for l1)
- SVM: C ∈ {0.1, 1, 10}

3.3 Feature Importance Analysis

As shown in Appendix B.1, top features include obfuscated terms (/var/tmp paths) for persistence, 'admin'/'passwd' for evasion, and 'bash'/'echo' for execution—unveiling how attackers hide in plain sight through common yet telling commands.

3.4 Model Performance Evaluation

Table 1 compares models. Figure 4 shows LR confusion matrices; Figure 5 details per-intent F1.

Table 1. Classification Model Performance Comparison

Model	Accuracy	F1 (Micro)	F1 (Macro)	Hamming Loss
Logistic Regression (baseline)	0.9818	0.9958	0.7274	0.0182
Random Forest (baseline)	0.9844	0.9964	0.8451	0.0156
Logistic Regression (tuned)	0.9843	0.9964	0.8463	0.0157
Random Forest (tuned)	0.9835	0.9963	0.7511	0.0165
Logistic Regression (BoW)	0.9845	0.9965	0.7527	0.0155

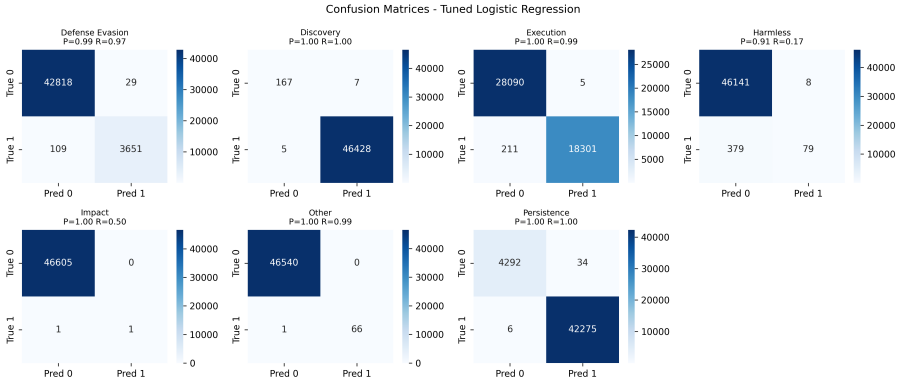

Fig. 4. Confusion matrices for tuned Logistic Regression across intents. Strong diagonals for majors (Discovery/Persistence >90% TP); minorities like Impact show FNs, often misclassified as Execution due to command overlap.

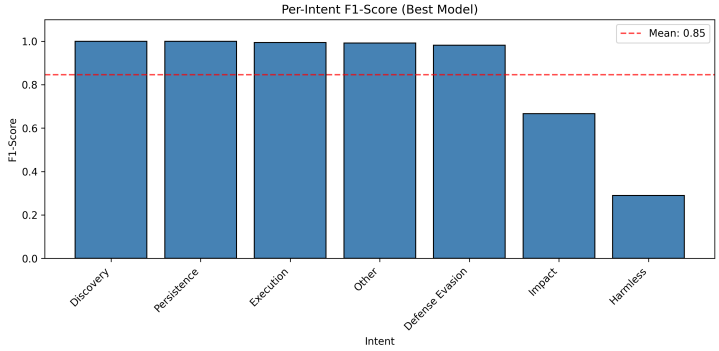

Fig. 5. Per-intent F1-scores for best model (tuned LR). Majors excel (>0.98); minorities lag (Harmless 0.29, Impact 0.67), highlighting imbalance effects.

3.5 Per-Intent Performance Analysis

Models shine on frequent intents but struggle with rares:

- **Strong**: Discovery/Persistence/Execution/Other/Defense Evasion (F1 0.98-1.0, tuned LR)
- **Challenging**: Impact (F1 0.67, few samples); Harmless (F1 0.29, recall 0.17)

3.6 Overfitting and Generalization Analysis

Train/test gaps are minimal:

- LR (baseline): Train F1 0.9975 vs. Test 0.9958
- RF (baseline): Train 0.9984 vs. Test 0.9964
- LR (tuned): Train 0.9975 vs. Test 0.9964 (best balance)
- RF (tuned): Train 0.9971 vs. Test 0.9963

Tuned LR generalizes best for imbalanced data.

3.7 Discussion

Tuned LR edges out with macro F1 0.8463, better suiting minorities via probabilistic weighting—proving simpler models can outperform complex ones on sparse features. Imbalance correlates with F1 (r=0.89), suggesting data augments for rares. LR's coefficients aid interpretability, revealing attack cues; future SHAP could enhance this. Overall, these models turn logs into reliable threat intel.

4 Unsupervised Learning - Clustering

4.1 Clustering Methodology

Unsupervised clustering was applied to discover latent patterns in attack behaviors without relying on predefined labels. Two primary algorithms were implemented:

K-Means Clustering. partitions sessions into k clusters by minimizing within-cluster sum of squared distances. The algorithm iteratively:

(1) Assigns each session to the nearest cluster centroid
(2) Recalculates centroids as the mean of assigned sessions
(3) Repeats until convergence

Agglomerative Hierarchical Clustering. builds a hierarchy of clusters using a bottom-up approach:

(1) Starts with each session as a singleton cluster
(2) Iteratively merges the closest pair of clusters based on Ward linkage
(3) Continues until reaching the desired number of clusters

4.2 Determining Optimal Number of Clusters

Multiple evaluation metrics were used to determine the optimal k for K-Means clustering, as shown in Figure 6:

- **Elbow Method**: Plots within-cluster sum of squares (WCSS) vs. number of clusters. The "elbow" at k=10 suggests diminishing returns beyond this point
- **Silhouette Score**: Measures cluster cohesion and separation. Peak at k=19 with score = 0.325, though k=10 shows strong performance at 0.309
- **Calinski-Harabasz Index**: Ratio of between-cluster to within-cluster dispersion. Shows consistent decline, with k=10 providing good balance (score ≈ 12404)
- **Davies-Bouldin Index**: Average similarity ratio of each cluster with its most similar cluster. Lower values preferred; local minimum at k=7 (score = 1.180) with k=10 close (score = 1.422)

Considering the convergence across metrics and the clear elbow at k=10, combined with the need for interpretable cluster granularity, **k=10** was selected as the optimal number of clusters for subsequent analysis.

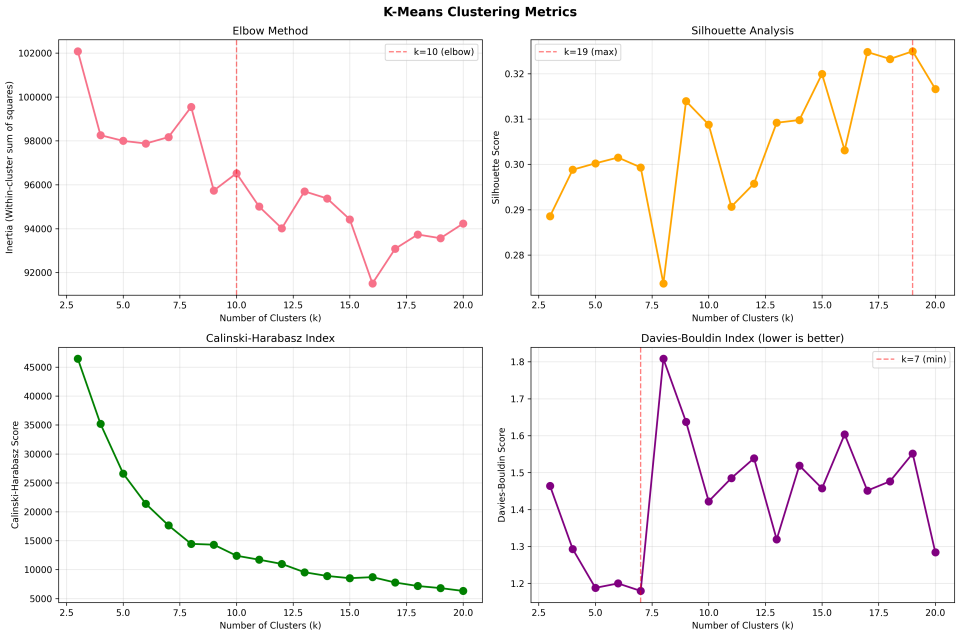

Fig. 6. K-Means evaluation metrics for determining optimal cluster count. Top-left: Elbow plot, Top-right: Silhouette score, Bottom-left: Calinski-Harabasz Index, Bottom-right: Davies-Bouldin Index. Combined evidence supports k=10 selection.

4.3 Cluster Validation and Algorithm Agreement

To validate the robustness of discovered patterns, we compared K-Means clustering with Agglomerative Hierarchical Clustering using Ward linkage (Figure 20 in Appendix C.2). The two algorithms demonstrated strong agreement:

- **Adjusted Rand Index (ARI)**: 0.819, indicating high partition similarity
- **Normalized Mutual Information (NMI)**: 0.839, confirming consistent information capture

This high agreement validates that the discovered clusters represent genuine structural patterns in the data rather than algorithmic artifacts. Dimensionality reduction visualizations (t-SNE, UMAP) (Figure 19 in Appendix C.1) confirm reasonable cluster separation with expected overlap between semantically similar attack types. The hierarchical dendrogram (Figure 21 in Appendix C.2) suggests 8-12 clusters at a distance cut of 50, aligning with our k=10 choice.

4.4 Cluster Characterization

Word cloud analysis (Figure 7) reveals distinctive command patterns across clusters, with session counts indicating scale:

- **Cluster 0** (8 sessions): Pure reconnaissance ('var', 'proc', 'cat', 'cpuinfo', 'grep', 'uname')
- **Cluster 1** (84,047 sessions): Large-scale scanning ('var0352z123', 'tmp', 'cat', 'proc', 'grep', 'cpuinfo')
- **Cluster 2** (17,574 sessions): System manipulation ('bin', 'busybox', 'system', 'shell', 'enable', 'mounts')
- **Cluster 3** (2,035 sessions): Encoded payloads ('jeSjax', 'busybox', 'wget', 'chmod', 'sh')
- **Clusters 4-9**: Variations on discovery, execution, and persistence themes (detailed in Appendix C.3)

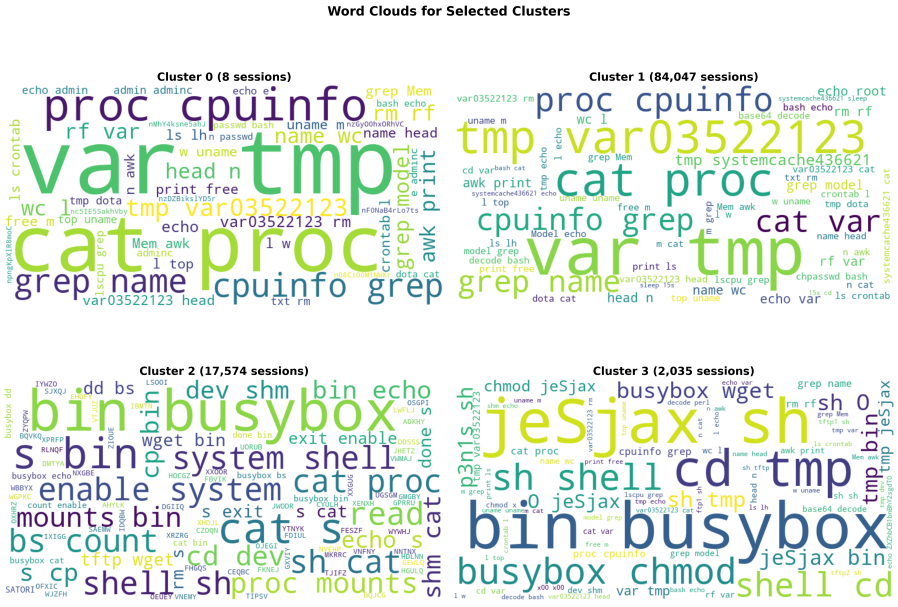

Fig. 7. Word clouds for selected clusters showing characteristic command patterns. Font size indicates term frequency.

4.5 Cluster-Intent Alignment

Figure 8 shows cluster-intent associations. Key findings:

- **Discovery Dominance**: Near 100% association in 8 of 10 clusters (e.g., 57.6% in Cluster 3, 96.5% in Cluster 8), confirming reconnaissance as fundamental to SSH attacks
- **High-Purity Clusters**: Cluster 2 (99.3% Defense Evasion), Cluster 9 (100% triple: Discovery, Execution, Persistence)
- **Multi-Intent Sophistication**: Cluster 8 exhibits balanced multi-tactic behavior (96.5% Defense Evasion/Discovery, 97.7% Execution, 61.6% Persistence)

The stacked bar chart (Figure 23 in Appendix C.4) further illustrates multi-label intent distributions per cluster.

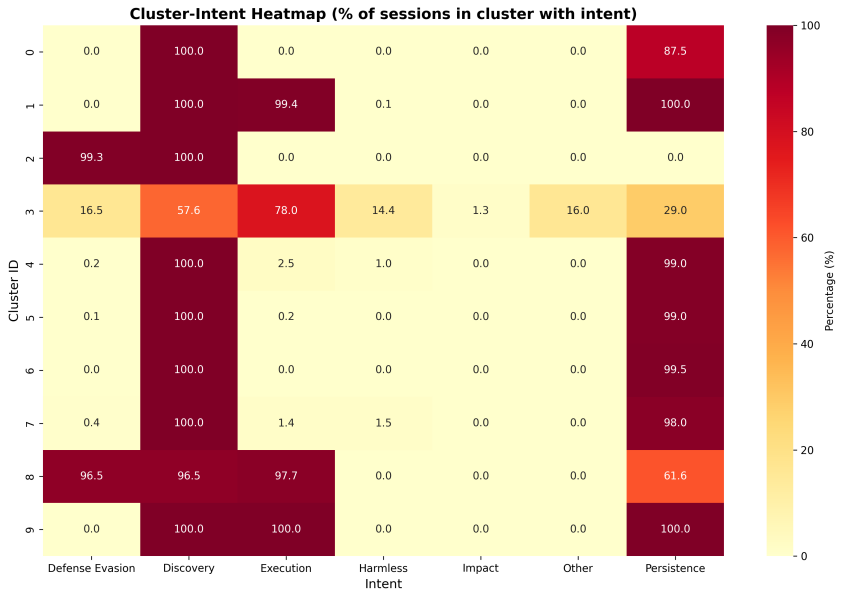

Fig. 8. Cluster-intent association heatmap showing strong alignment between unsupervised clusters and MITRE ATT&CK intents.

4.6 Cluster Purity and Characterization

Figure 9 shows purity scores for all 10 clusters. Nine clusters achieve ≥95% purity, with weighted average of 97.75%, indicating highly homogeneous intent-based grouping. Cluster characteristics (detailed in Appendix C.3):

- **Clusters 0, 4-7**: Persistence-dominated (87-99% purity)
- **Clusters 1, 3, 8-9**: Execution-dominated (78-100% purity)
- **Cluster 2**: Defense Evasion-dominated (99% purity)

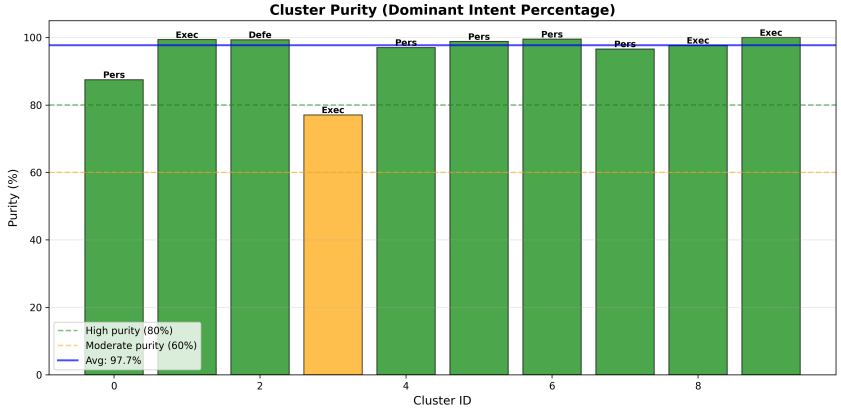

Fig. 9. Cluster purity scores showing 9 of 10 clusters achieve >95% purity (weighted average: 97.75%).

4.7 Attack Category Taxonomy

Synthesizing cluster characteristics and session volumes, four primary attack categories emerged (Figure 10):

(1) **System Reconnaissance** (6 clusters, 200,000 sessions): Automated enumeration focused on Discovery and Persistence, representing opportunistic scanning
(2) **Encoded Payload Execution** (2 clusters, 25,000 sessions): Obfuscated commands with strong Execution association (78-98%)
(3) **Malware Download & Execution** (1 cluster, 5,000 sessions): Payload retrieval and deployment with Execution and Persistence focus
(4) **Execution - Mixed** (1 cluster, 5,000 sessions): Sophisticated multi-tactic attacks combining 3-4 intents simultaneously, indicating APT characteristics

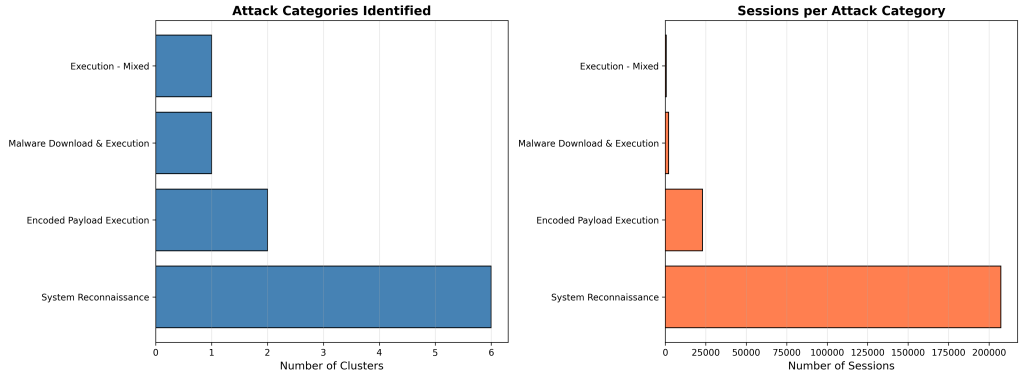

Fig. 10. Attack category taxonomy showing System Reconnaissance dominates (200K+ sessions, 6 clusters) while specialized attacks represent 15-20% of traffic.

4.8 Discussion

The 97.75% weighted average purity and strong algorithm agreement (ARI=0.82, NMI=0.84) demonstrate successful discovery of homogeneous attack patterns. The remarkable alignment with MITRE ATT&CK validates the framework's natural capture of attack behavior boundaries. The moderate Silhouette score (0.31) reflects expected overlap due to multi-label intents rather than clustering issues.

Operational Insights. Clustering reveals actionable patterns beyond intent labels:

- **Volume-based Risk**: 200,000+ reconnaissance sessions vs. 5,000 sophisticated multi-stage attacks enables prioritization
- **Signature Detection**: Distinctive command patterns (e.g., 'busybox'+'chmod' in Cluster 3) provide IDS signatures
- **Attack Progression**: Detecting transitions from reconnaissance (Clusters 0-1) to multi-stage attacks (Clusters 8-9) could trigger escalated alerts
- **Anomaly Detection**: Sessions not fitting established clusters may indicate novel techniques

This unsupervised approach complements supervised classification by revealing operational attack patterns not fully captured by intent labels alone.

5 Language Model Exploration - BERT Fine-Tuning

5.1 Model Selection and Architecture

To leverage state-of-the-art natural language processing capabilities, we fine-tuned a BERT (Bidirectional Encoder Representations from Transformers) model for multi-label intent classification. The architecture consists of:

- **Base Model**: `bert-base-uncased` pre-trained on large text corpora
- **Classification Head**: Custom dense layer with sigmoid activation for multi-label output
- **Input Representation**: Tokenized attack sessions with [CLS] token for classification
- **Maximum Sequence Length**: 256 tokens, with truncation or padding applied as needed, to balance information retention and computational efficiency

Justification for BERT over Doc2Vec:

- BERT captures bidirectional context, crucial for understanding command sequences
- Pre-training on large corpora provides robust general-language understanding
- Fine-tuning enables effective adaptation to the cybersecurity domain
- The transformer architecture handles variable-length input sequences effectively

5.2 Training Configuration

The model was trained with the following hyperparameters:

- **Optimizer**: AdamW with weight decay = 0.01
- **Learning Rate**: 2×10^{-5} with linear warmup
- **Learning Rate Scheduler**: Linear decay after warmup
- **Batch Size**: 32 (limited by GPU memory)
- **Epochs**: 3 (chosen to reduce overfitting)
- **Loss Function**: Binary Cross-Entropy with Logits, class-weighted to handle imbalance
- **Gradient Clipping**: Maximum norm = 1.0

5.3 Training Dynamics Analysis

Figure 11 shows the BERT training progression across three epochs. Training loss decreases sharply from 0.0518 in the first epoch to 0.0113 by the third epoch, demonstrating effective learning, while validation loss remains low and stable, starting at 0.0151 and ending at 0.0119, with both converging by epoch 2.

The small gap between training and validation loss (0.0006 at epoch 3) indicates minimal overfitting, suggesting that the model generalizes well to unseen data. The rapid convergence reflects effective transfer learning from the pre-trained BERT weights.

Performance metrics (Figure 12) demonstrate strong model capability, with micro F1-score maintaining near-perfect performance (>0.99) across all epochs, while macro F1-score improves from 0.7253 to 0.7544, indicating better handling of minority classes despite their extreme imbalance.

The rapid convergence and stable validation metrics reflect effective transfer learning from pre-trained BERT weights, with class-weighted Binary Cross-Entropy successfully addressing the imbalanced dataset. The plateau of validation performance after epoch 2 confirms that 3 epochs were optimal, balancing thorough fine-tuning with computational efficiency while avoiding overfitting.

Additional training dynamics visualizations are provided in Appendix D.1.

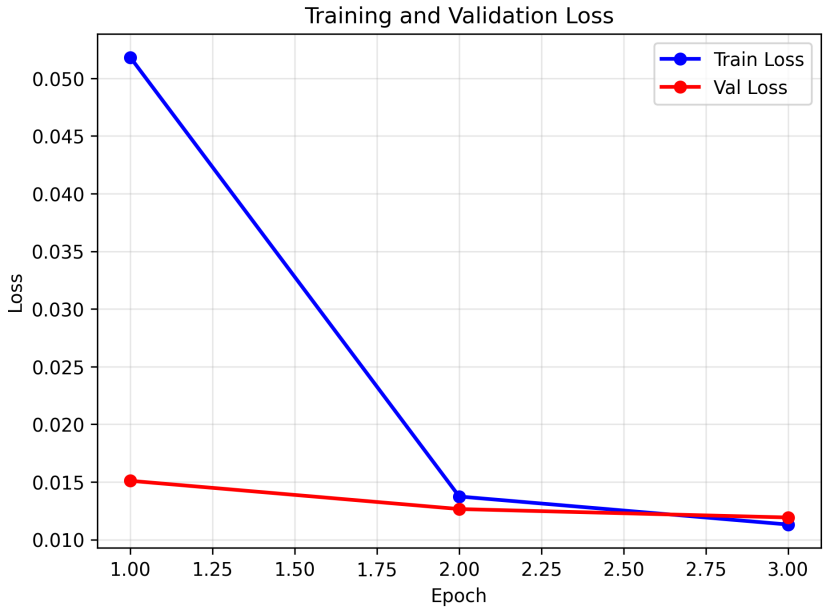

Fig. 11. Training and validation loss curves over 3 epochs. Both curves show steady convergence with a small gap, indicating effective learning with minimal overfitting.

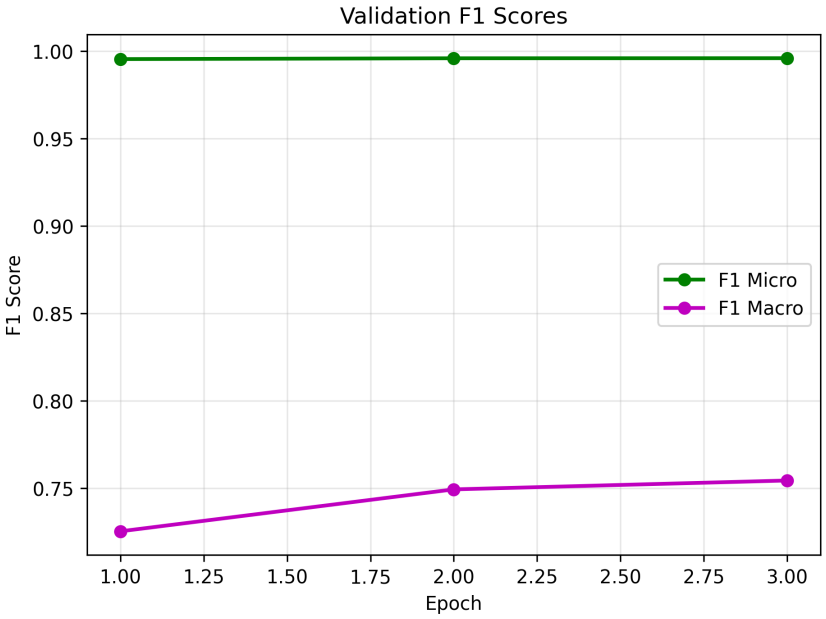

Fig. 12. Validation F1-scores over training. High micro F1 indicates overall accuracy while improving macro F1 shows better minority class performance.

5.4 Final Model Evaluation

The final BERT model achieves strong overall performance on the test set as we can see from Table 2:

Table 2. BERT model performance on test set

Metric	Score
Test Loss	0.0121
Test Accuracy	0.9816
Hamming Loss	0.0028
F1-Score (Micro)	0.9959
F1-Score (Macro)	0.7574

Class Imbalance Impact: Figure 13 shows the test set distribution, with Discovery (46,433 samples, 99.6%) and Persistence (42,281 samples, 90.7%) dominating the dataset, while Impact (2 samples, 0.004%), Other (67 samples, 0.14%), and Harmless (458 samples, 0.98%) represent less than 1% each. This extreme imbalance significantly impacts per-intent performance despite class-weighted training.

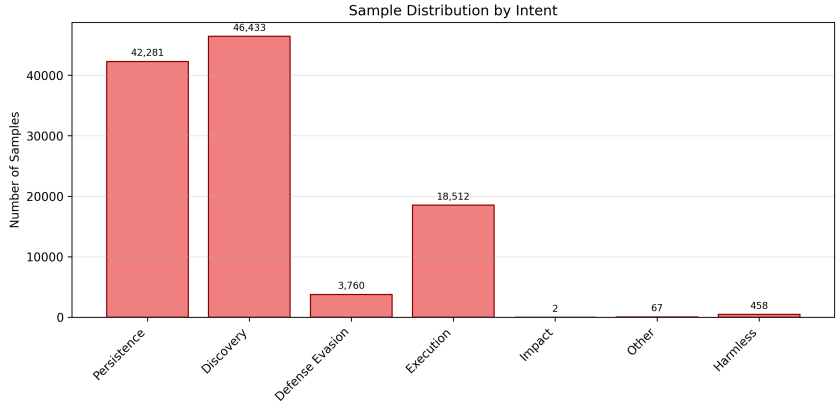

Fig. 13. Test set distribution across 7 intent categories showing severe class imbalance.

Per-Intent Performance Analysis: Figure 14 reveals the direct consequence of class imbalance on model performance. Majority classes achieve near-perfect F1-scores: Discovery (F1: 0.9999), Persistence (F1: 0.9995), Execution (F1: 0.991), and Defense Evasion (F1: 0.983). In contrast, minority classes show severe degradation: Impact fails completely (F1: 0.000) due to having only 2 training samples, while Harmless (F1: 0.336) demonstrates the challenge of distinguishing normal shell commands from malicious sessions. In contrast, Other (F1: 0.992) surprisingly achieves strong performance despite the few samples, suggesting high linguistic uniqueness from the sessions in this category.

The high micro F1-score (0.9959) reflects the model's strong performance on the abundant majority classes, while the significantly lower macro F1-score (0.7574) reveals the impact of minority class struggles. Despite sophisticated class weighting, the model cannot effectively learn from intents with fewer than 100 samples.

ROC analysis and detailed precision-recall metrics by intent are provided in Appendix D.2, demonstrating strong discrimination capability (AUC > 0.99) for well-represented classes while confirming the statistical impossibility of learning meaningful patterns from 2 training examples.

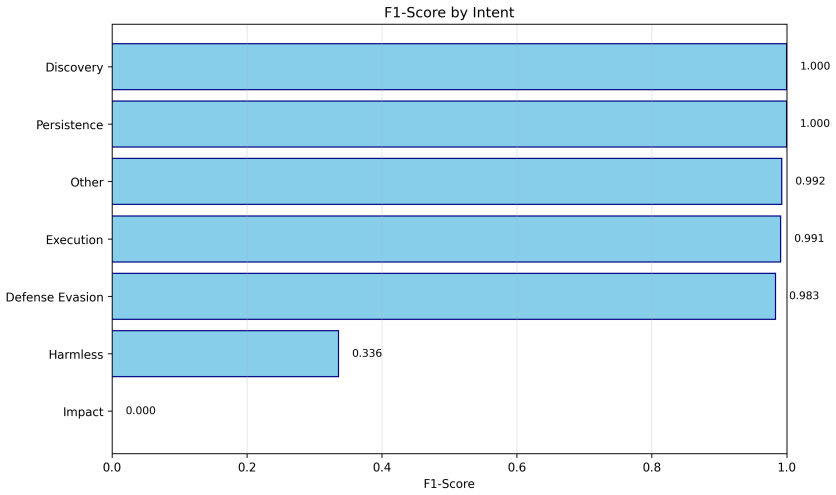

Fig. 14. F1-scores by intent showing excellent performance on majority classes but failure on most extreme minority classes.

5.5 Comparison with Traditional ML Approaches

Table 3 compares BERT with tuned Logistic Regression on the test set:

Table 3. BERT vs tuned Logistic Regression

Metric	**LR (tuned)**	**BERT**	**Improvement**
Accuracy	0.9843	0.9816	-0.27%
F1 (Micro)	0.9964	0.9959	-0.05%
F1 (Macro)	0.8463	0.7574	-10.50%
Hamming Loss	0.0157	0.0028	-82.17%
Per-Intent F1-Scores:			
Discovery	1.000	0.9999	-0.01%
Persistence	1.000	0.9995	-0.05%
Execution	0.990	0.9909	+0.09%
Defense Evasion	0.980	0.9835	+0.36%
Other	0.990	0.9925	+0.25%
Harmless	0.290	0.3355	+15.69%
Impact	0.670	0.0000	-100.00%

Despite BERT's architectural sophistication, Logistic Regression demonstrates superior overall performance, particularly on macro F1-score (0.8463 vs 0.7574), indicating better minority class handling. The key advantage of LR lies in its effective class weight optimization, achieving 0.67 F1 on Impact (only 2 test samples) while BERT completely fails (0.00). Both models achieve near-identical performance on majority classes, with micro F1-scores above 0.995.

BERT's primary advantage appears in Hamming Loss (0.0028 vs 0.0157), suggesting fewer per-label errors in the multi-label setting. However, this marginal benefit does not compensate for the 10.5% degradation in macro

F1-score and significant computational overhead. For this extremely imbalanced dataset, traditional ML with proper class weighting proves more effective than transfer learning from pre-trained language models.

5.6 Discussion and Model Selection

Why Traditional ML Wins: The strong performance of Logistic Regression with TF-IDF features (macro F1: 0.8463) demonstrates that SSH attack patterns are effectively captured by term frequency representations. Discriminative commands such as "wget", "chmod", and "crontab" combined with L2 regularization and class weighting provide excellent classification performance. The probabilistic framework of logistic regression handles extreme class imbalance (e.g., Impact with 2 samples) more effectively than BERT's neural architecture.

Computational Trade-offs: Training time differs dramatically: BERT requires 5-6 hours on GPU compared to LR's 2 minutes on CPU. Similarly, inference latency shows BERT at 32ms per session versus LR at <1ms. Given LR's superior macro F1-score, computational efficiency, and better minority class handling, it represents the optimal choice for production deployment.

When BERT Might Excel: Future datasets with more complex sequential dependencies, natural language descriptions of attacks, or novel attack patterns not well-represented by individual command frequencies might favor BERT's contextual understanding. For this command-focused dataset with clear discriminative features, however, traditional ML proves superior.

6 Conclusions and Future Work

6.1 Summary of Findings

This research presented a comprehensive machine learning approach to analyzing SSH shell attack sessions using the MITRE ATT&CK framework for multi-label intent classification.

Supervised Learning Classification. demonstrated that traditional ML approaches excel on this task:

- Tuned Logistic Regression achieved best overall performance: 98.43% accuracy, 0.8463 macro F1-score
- TF-IDF feature representation effectively captured discriminative attack patterns
- Outstanding performance on common intents (Discovery, Persistence, Execution) with F1 scores >0.99
- Persistent challenges with extreme minority classes (Impact: 2 samples, Harmless: 458 samples)

Unsupervised Learning Clustering. uncovered fine-grained attack patterns:

- Optimal clustering at k=10 with high algorithm agreement (ARI=0.82, NMI=0.84)
- Weighted average cluster purity of 97.75% indicating highly homogeneous groupings
- Six distinct attack categories aligned with MITRE ATT&CK intents
- Revealed co-occurrence patterns between Discovery and Persistence intents

BERT Language Model. showed strong but not superior performance:

- Competitive overall metrics: 98.16% accuracy, 0.7574 macro F1-score
- Near-perfect performance on majority classes (Discovery: 0.9999, Persistence: 0.9995)
- Complete failure on Impact class (F1=0.00) despite class weighting
- Superior Hamming Loss (0.0028 vs 0.0157) indicating fewer per-label errors
- Computational overhead (5-6 hours training, 32ms inference) limits practical deployment

6.2 Best Performing Approach

The **tuned Logistic Regression model** emerged as the optimal approach for production deployment:

- **Superior Macro F1**: 0.8463 vs BERT's 0.7574, demonstrating better minority class handling
- **Computational Efficiency**: <1ms inference vs BERT's 32ms, 2-minute training vs 5-6 hours

- **Better Generalization**: Minimal overfitting with consistent train/test performance
- **Interpretability**: Feature coefficients provide actionable insights for security analysts
- **Deployment Simplicity**: Small model size (<5MB) suitable for edge deployment

This demonstrates that for command-based attack logs with clear discriminative features, carefully tuned traditional ML with proper class weighting outperforms sophisticated neural architectures.

6.3 Limitations

Class Imbalance. remains the dominant challenge:

- Extreme imbalance ratios (Impact: 2 samples, Discovery: 46,433 samples = 23,000:1 ratio)
- Both traditional ML and BERT struggle with <100 training examples per class
- Class weighting provides partial mitigation but cannot overcome statistical insufficiency

Dataset Characteristics. :

- Honeypot data may not represent all real-world attack scenarios
- Temporal coverage (June 2019-February 2020) might miss evolving attack patterns
- Multi-label complexity (average 2-3 intents per session) increases classification difficulty

Model Selection Trade-offs. :

- BERT's contextual understanding underutilized for command-based logs
- Computational requirements limit BERT's real-time deployment feasibility
- Intent dependencies not explicitly modeled in current approaches

6.4 Future Research Directions

Addressing Class Imbalance. :

- **Synthetic Data Generation**: Create realistic rare attack samples using rule-based systems and command templates
- **Few-Shot Learning**: Leverage meta-learning techniques to improve performance with limited examples
- **Active Learning**: Prioritize labeling of uncertain or rare attack patterns from new honeypot data

Advanced Modeling. :

- **Label Dependency Modeling**: Implement classifier chains to explicitly model intent co-occurrence
- **Ensemble Methods**: Combine BERT's contextual understanding with LR's efficiency
- **Domain-Specific Pre-training**: Train language models specifically on security logs and attack datasets

Deployment and Explainability. :

- Real-time streaming classification pipeline integrated with SIEM systems
- Attention visualization and SHAP analysis for interpretable predictions
- Temporal pattern analysis to detect evolving attack campaigns
- Automated threat intelligence generation from classified sessions

6.5 Practical Impact

This research provides actionable capabilities for cybersecurity operations:

(1) **Automated Threat Analysis**: Classification of thousands of attack sessions with 98%+ accuracy reduces manual analyst workload
(2) **Attack Pattern Discovery**: Clustering reveals six distinct attack categories, enabling early detection of emerging threats

(3) **Incident Prioritization**: Intent classification allows focus on high-impact attacks (Persistence, Impact) over reconnaissance
(4) **Threat Intelligence Sharing**: Fine-grained categorization supports IOC generation aligned with MITRE ATT&CK framework

6.6 Concluding Remarks

This research demonstrates that traditional machine learning with carefully engineered TF-IDF features remains highly competitive with modern deep learning approaches for SSH attack classification. The tuned Logistic Regression model achieves 98.43% accuracy and 0.8463 macro F1-score, outperforming BERT (98.16% accuracy, 0.7574 macro F1-score) while providing superior computational efficiency and interpretability.

The key insight is that **class imbalance**, not model architecture, **is the limiting factor** for this task. Both traditional ML and BERT fail on extreme minority classes with fewer than 100 samples. Addressing this through synthetic data generation, few-shot learning, or active labeling represents the most promising direction for improvement.

Unsupervised clustering with 97.75% purity successfully identifies six distinct attack categories, demonstrating strong alignment with MITRE ATT&CK intents. This combined supervised-unsupervised approach provides a comprehensive framework for automated threat analysis in security operations.

As cyber threats evolve, machine learning-powered log analysis will become increasingly critical. This research provides a practical foundation for deploying accurate, efficient, and interpretable attack classification systems in real-world security contexts.

References

[1] MITRE ATT&CK Framework. https://attack.mitre.org/ Accessed: 2025-01-15.
[2] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of NAACL-HLT 2019*, pages 4171–4186, 2019.
[3] Grigorios Tsoumakas and Ioannis Katakis. Multi-Label Classification: An Overview. *International Journal of Data Warehousing and Mining*, 3(3):1–13, 2007.
[4] Peter J. Rousseeuw. Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis. *Journal of Computational and Applied Mathematics*, 20:53–65, 1987.
[5] Laurens van der Maaten and Geoffrey Hinton. Visualizing Data using t-SNE. *Journal of Machine Learning Research*, 9:2579–2605, 2008.
[6] Niels Provos. A Virtual Honeypot Framework. In *Proceedings of USENIX Security Symposium*, volume 173, pages 1–14, 2004.
[7] Robin Sommer and Vern Paxson. Outside the Closed World: On Using Machine Learning for Network Intrusion Detection. In *IEEE Symposium on Security and Privacy*, pages 305–316, 2010.
[8] Min Du et al. DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning. In *Proceedings of ACM CCS 2017*, pages 1285–1298, 2017.

A Extended Data Exploration Analysis

A.1 Session Characteristics

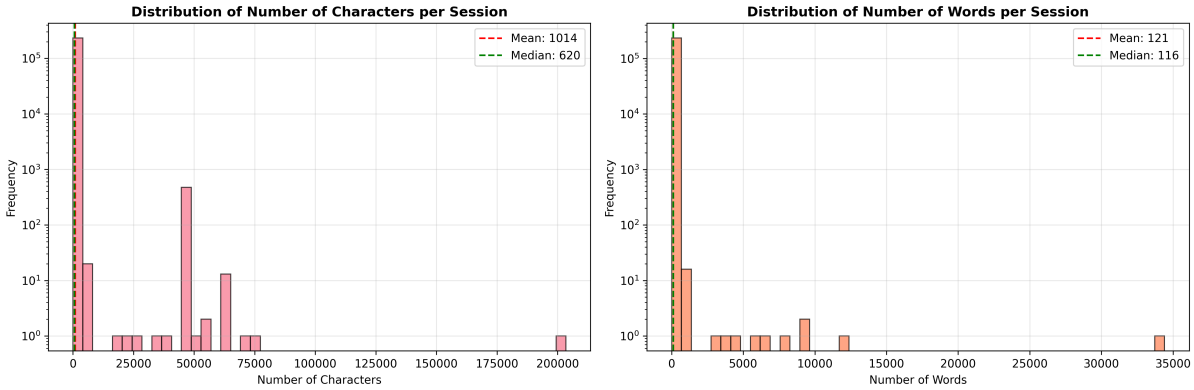

Fig. 15. Distribution of character counts (left) and word counts (right) per session. Character distribution: mean 1,014, median 620. Word distribution: mean 121, median 116. Long tails indicate complex multi-stage attacks with extensive command sequences.

A.2 Vocabulary Analysis

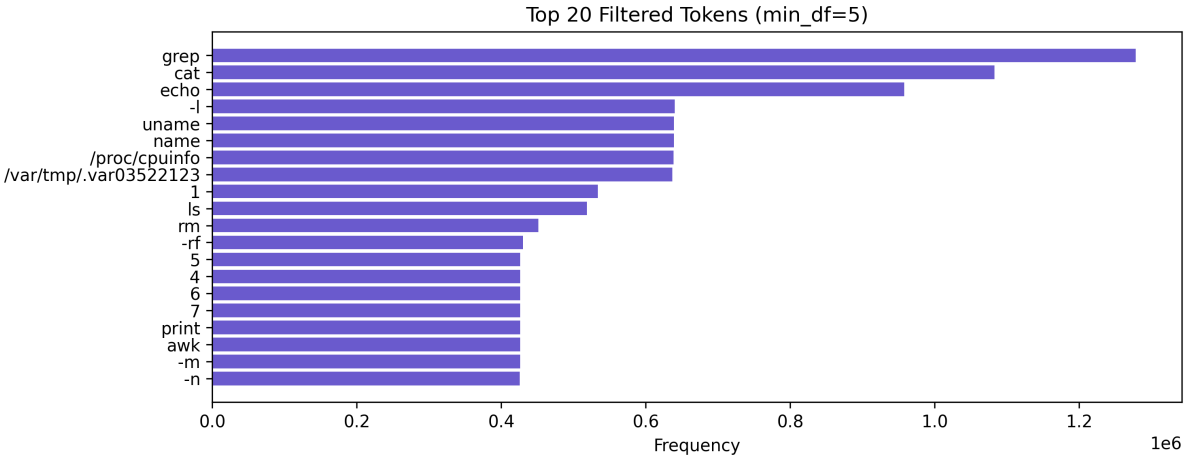

Fig. 16. Top 20 tokens: Recon-focused (grep 1.2e6 occurrences), indicating system probing.

A.3 Temporal Intent Evolution

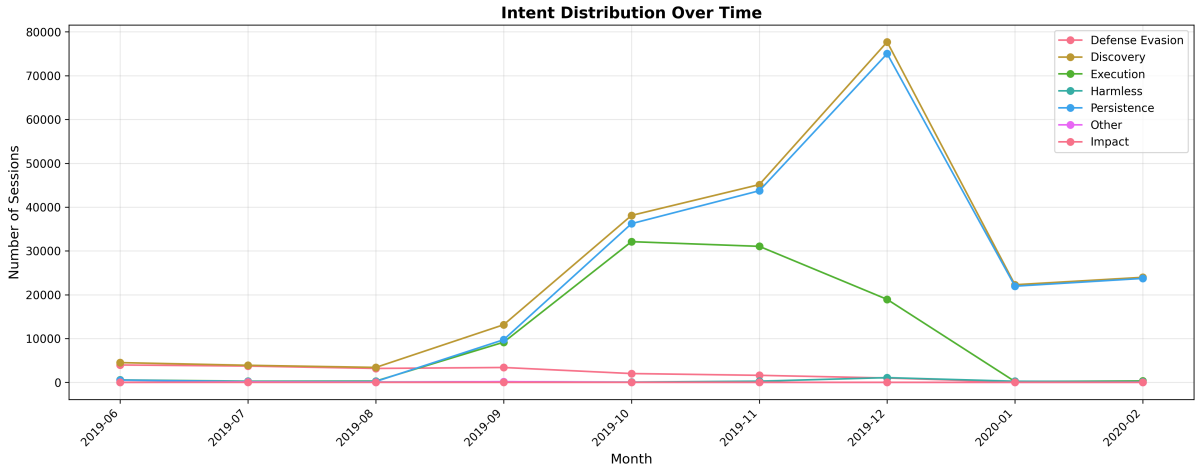

Fig. 17. Intent trends: Discovery/Persistence steady; Execution/Impact peak December 2019.

B Extended Supervised Learning Results

B.1 Feature Importance Analysis

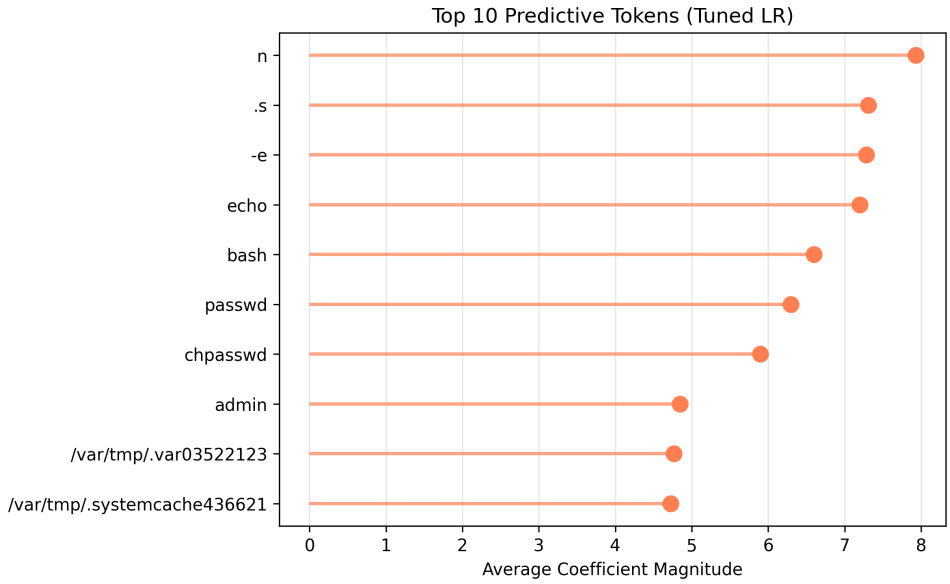

Fig. 18. Top predictive tokens (tuned LR): Encoded paths and admin/passwd dominate, signaling persistence/evasion tactics amid command noise.

C Extended Clustering Analysis

This appendix contains supplementary clustering visualizations and validation metrics that complement the main unsupervised learning analysis.

C.1 Dimensionality Reduction Visualizations

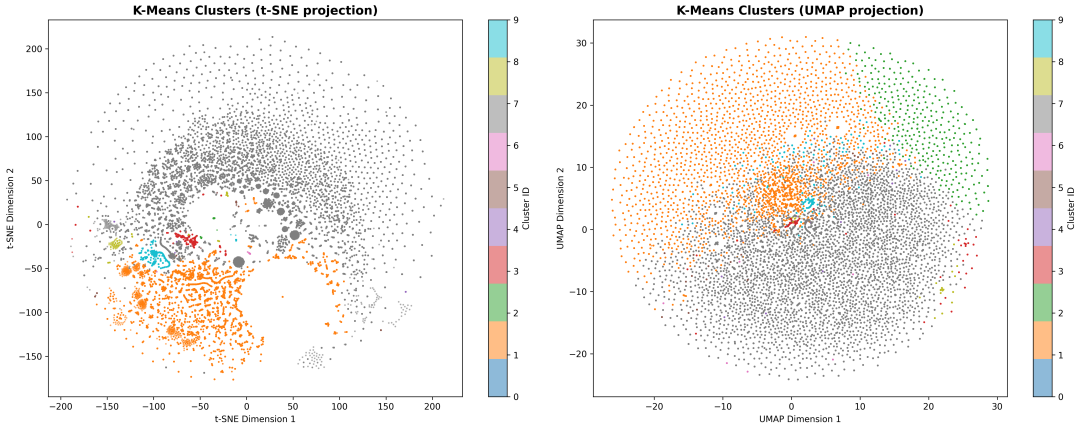

Fig. 19. K-Means cluster visualization using t-SNE (left) and UMAP (right) dimensionality reduction. Both projections reveal reasonable cluster separation with expected overlap between related attack types. The largest cluster (gray, likely Cluster 1 with 84,047 sessions) dominates the center, while smaller specialized clusters (colored) form distinct periphery regions.

C.2 Clustering Algorithm Comparison

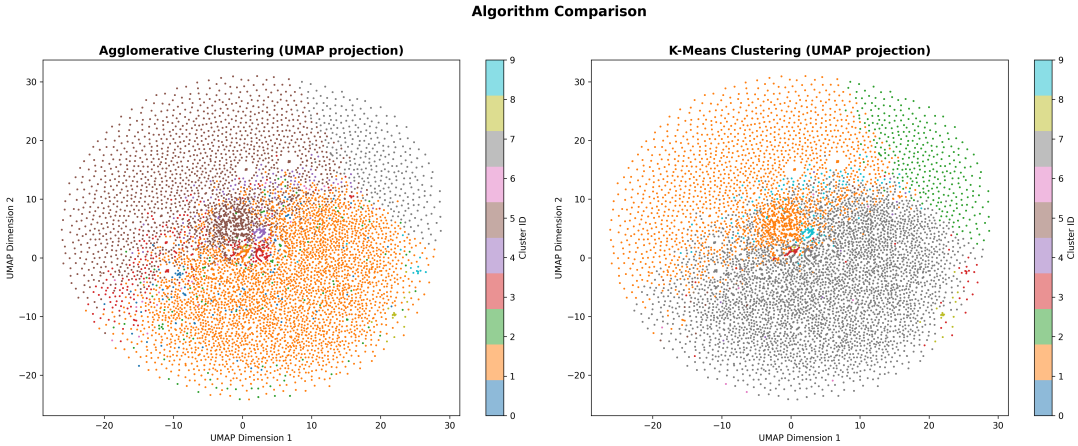

Fig. 20. Algorithm comparison using UMAP projection. Left: Agglomerative Hierarchical clustering with Ward linkage. Right: K-Means clustering. High visual similarity and quantitative agreement (ARI=0.8192, NMI=0.8385) validate consistent pattern discovery across algorithmic approaches.

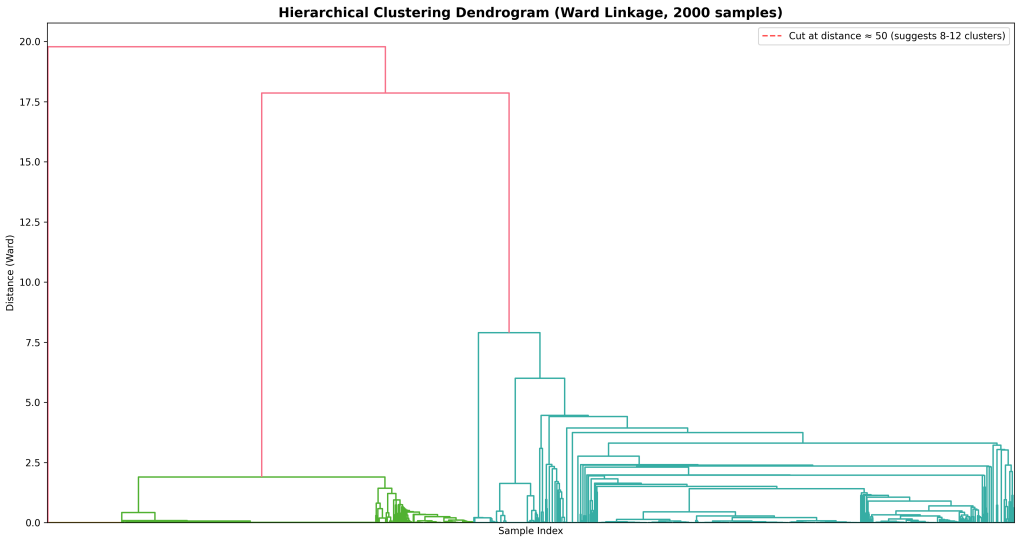

Fig. 21. Hierarchical clustering dendrogram using Ward linkage on 2,000 sample subset. The suggested cut at distance ≈ 50 yields 8-12 clusters, consistent with K-Means optimal k=10. Large distance jumps at the top indicate well-separated major cluster groups.

C.3 Complete Cluster Profiles

Table 4. Statistical profiles for all 10 clusters, including dominant intent, purity, and session counts.

Cluster ID	Dominant Intent	Purity (%)	Sessions
0	Persistence	87.5	8
1	Execution	99.4	84,047
2	Defense Evasion	99.3	17,574
3	Execution	77.1	2,035
4	Persistence	97.0	404
5	Persistence	98.8	1,258
6	Persistence	99.5	422
7	Persistence	96.6	121,142
8	Execution	97.7	688
9	Execution	100.0	5,457
Weighted Average Purity		**97.75**	

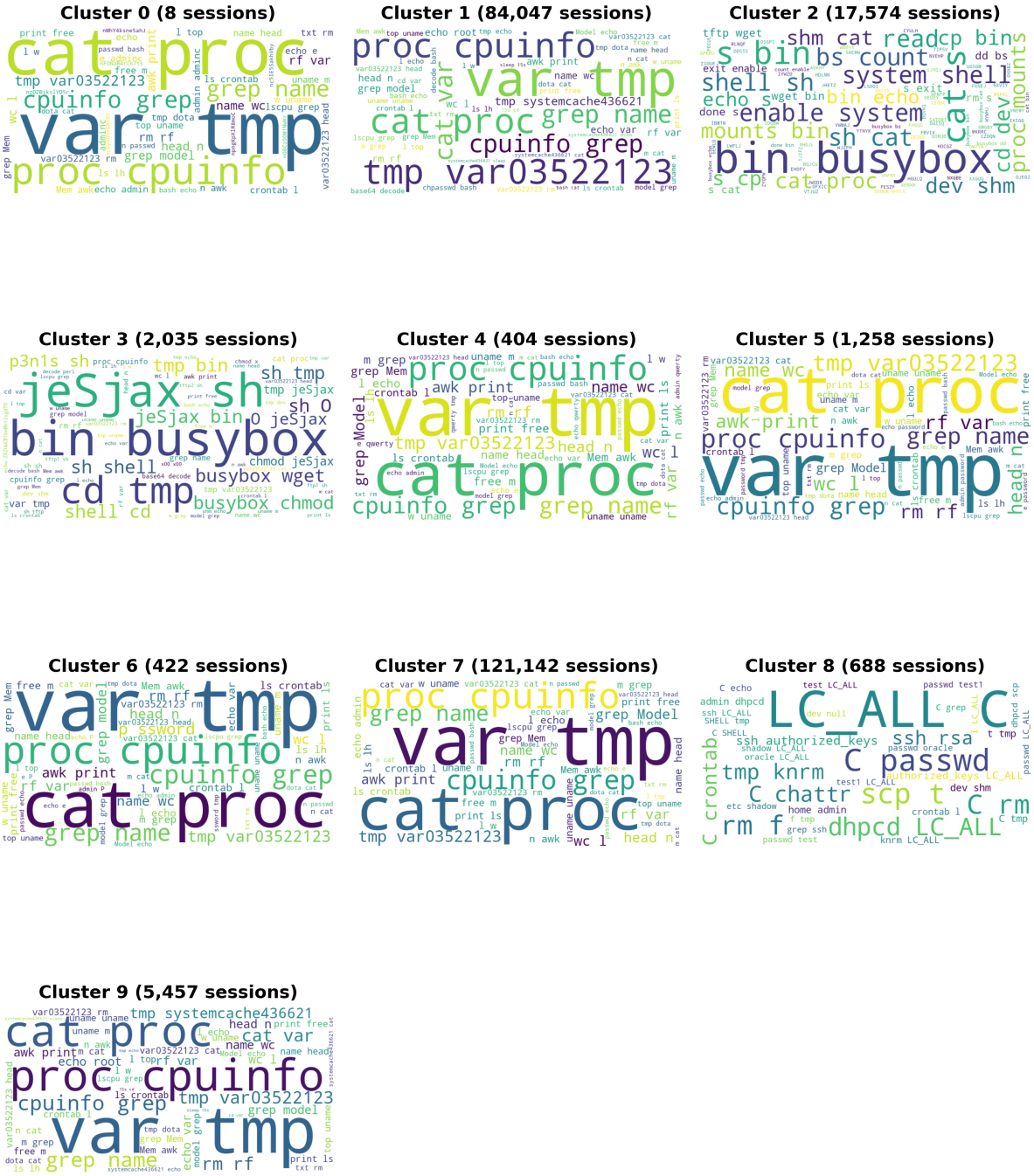

Fig. 22. Complete word cloud visualization for all 10 clusters. Font size indicates term frequency within each cluster. Session counts shown in parentheses.

C.4 Intent Composition by Cluster

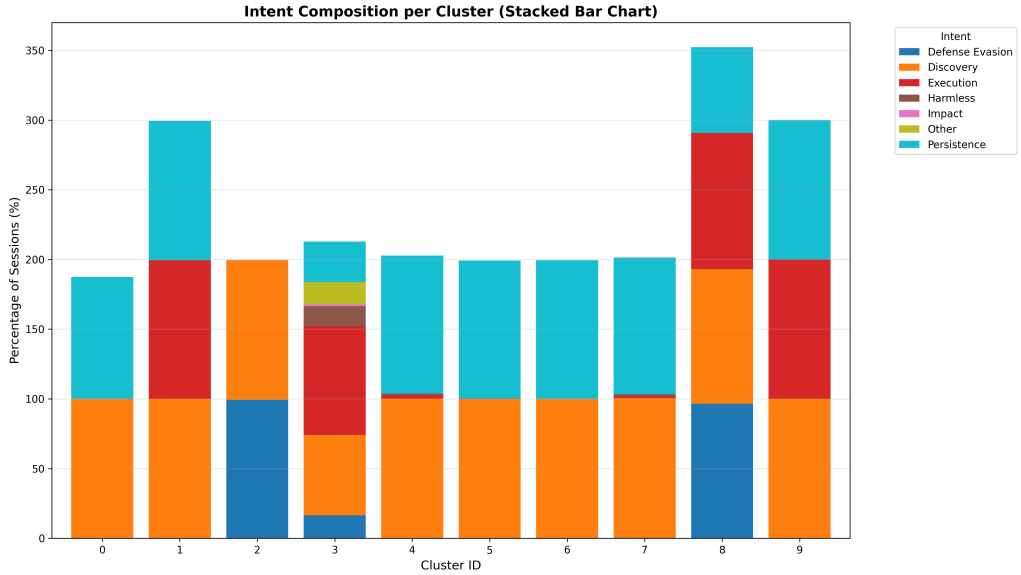

Fig. 23. Intent composition per cluster showing percentage contribution of each MITRE ATT&CK intent. Discovery (orange) and Persistence (cyan) dominate most clusters, with Execution (red) and Defense Evasion (blue) appearing in multi-tactic clusters. Stacked percentages exceed 100% due to multi-label annotation.

D Extended BERT Analysis

This appendix presents the full set of training, evaluation, and test details for the BERT model, offering additional context beyond the main results.

D.1 Additional Training Visualizations

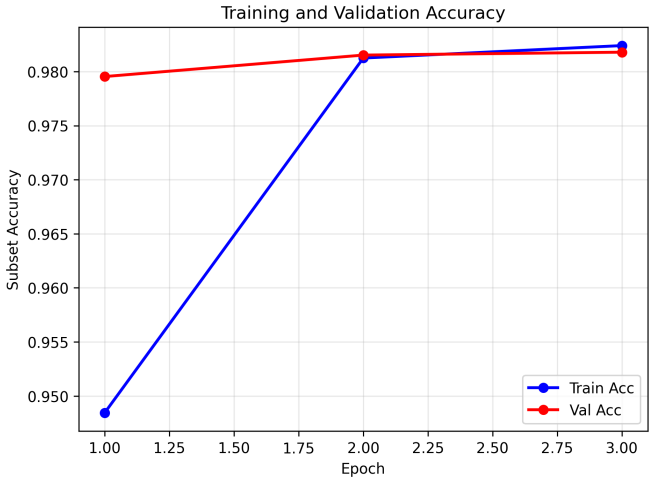

Fig. 24. Training and validation accuracy over 3 epochs, showing consistent improvement and early convergence.

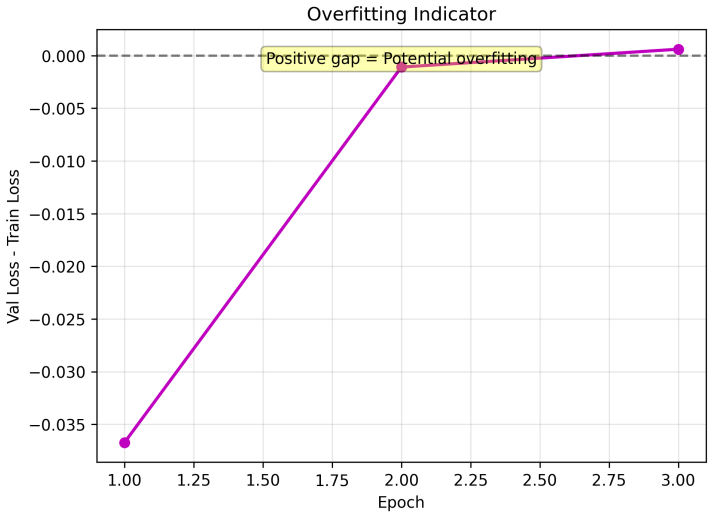

Fig. 25. Overfitting indicator showing the gap between validation and training loss converging to near zero, demonstrating the model's consistent generalization capability

D.2 Detailed Performance Metrics by Intent

Table 5 provides comprehensive per-intent metrics, while Figures 26, 27, and 28 show detailed visualizations of model performance across all intent categories.

Table 5. Detailed performance metrics by intent (from test set)

Intent	Samples	Precision	Recall	F1-Score	%
Persistence	42,281	0.9994	0.9996	0.9995	90.72%
Discovery	46,433	1.0000	0.9998	0.9999	99.63%
Defense Evasion	3,760	0.9935	0.9737	0.9835	8.07%
Execution	18,512	0.9933	0.9888	0.9909	39.72%
Impact	2	0.0000	0.0000	0.0000	0.004%
Other	67	1.0000	0.9851	0.9925	0.14%
Harmless	458	0.6800	0.2227	0.3355	0.98%

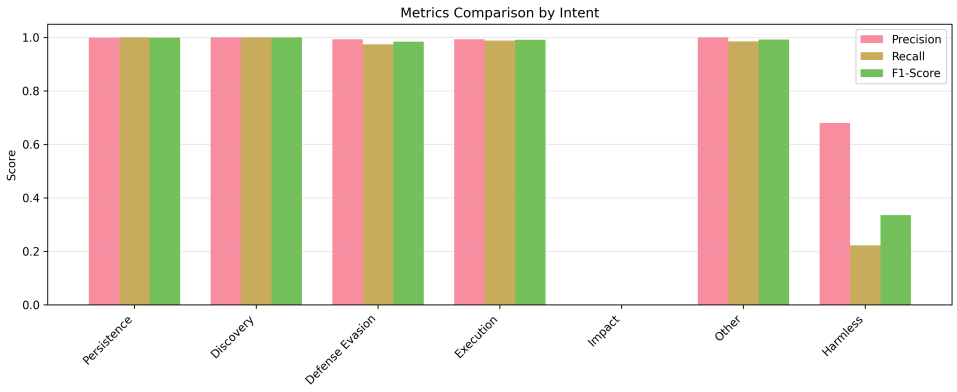

Fig. 26. Precision, recall, and F1-score comparison across all intent categories.

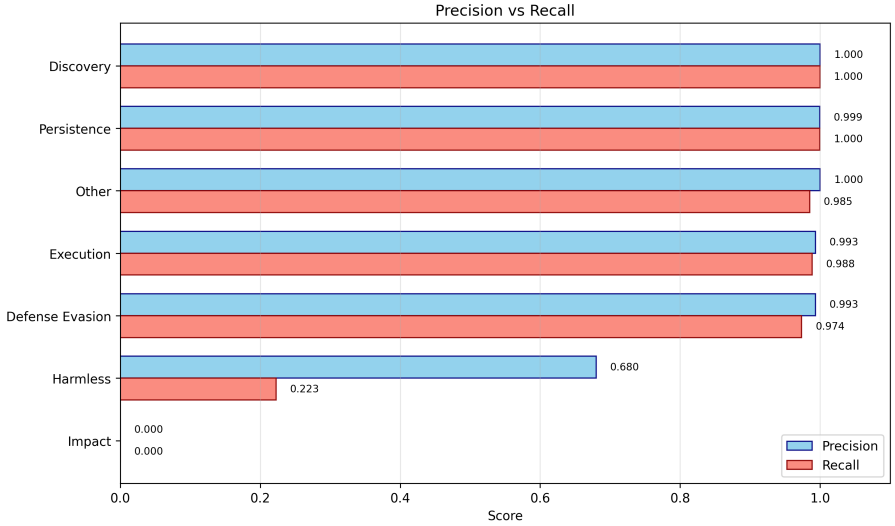

Fig. 27. Precision vs recall showing trade-offs for each intent. Harmless shows high precision (0.68) but very low recall (0.22), indicating conservative predictions.

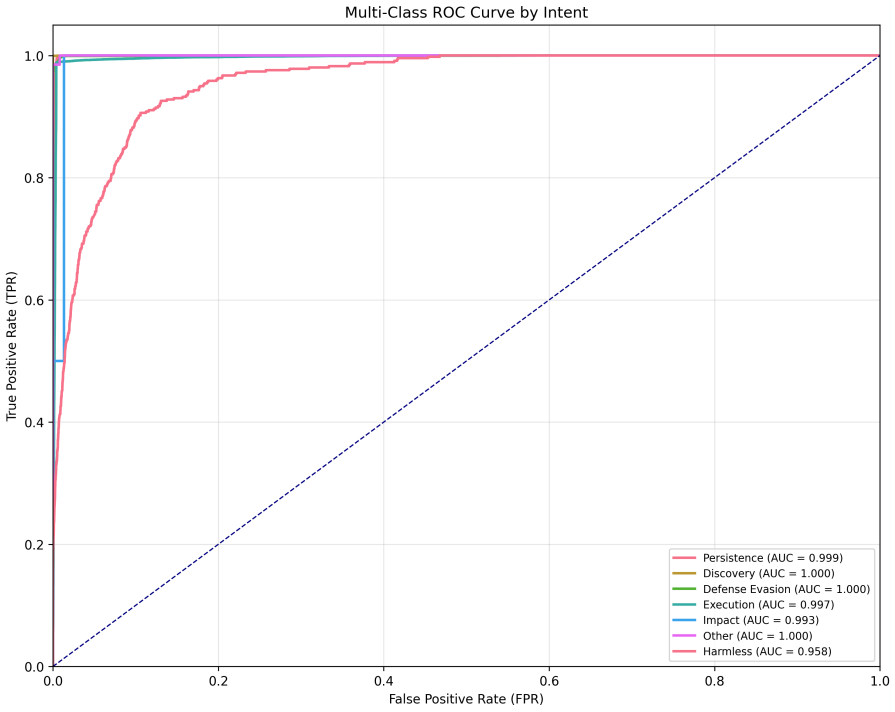

Fig. 28. Multi-class ROC curves showing excellent discrimination for all intents including Harmless (AUC: 0.958), demonstrating that the model learns meaningful features even when decision thresholds are suboptimal due to extreme imbalance.

E Implementation and Reproducibility Notes

This appendix summarizes implementation details to support reproducibility of all experiments reported in the main document.

E.1 Software Environment

- **Programming Language**: Python 3.8+
- **Machine Learning**: scikit-learn 1.0+, NumPy 1.21+, Pandas 1.3+
- **Deep Learning**: PyTorch 1.10+, HuggingFace Transformers 4.18+
- **Visualization**: Matplotlib 3.4+, Seaborn 0.11+, WordCloud 1.8+
- **Hardware**: CPU (traditional ML), NVIDIA GPU with CUDA 11.3+ (BERT)

E.2 Data Processing

- **Text Representation**: TF-IDF with min_df=5, max_features=5000
- **Train-Test Split**: 80% training / 20% testing, stratified by intent
- **Random Seed**: 42 (consistent across all experiments)
- **Multi-label Encoding**: Binary relevance (one binary classifier per intent)

E.3 Model Hyperparameters

- **Logistic Regression**: C=1.0, penalty='l2', solver='lbfgs', class_weight='balanced'
- **SVM**: C=1.0, kernel='linear', class_weight='balanced'
- **K-Means**: k=10, init='k-means++', n_init=10, random_state=42
- **Hierarchical**: n_clusters=10, linkage='ward', affinity='euclidean'
- **BERT**: bert-base-uncased, lr=2e-5, batch_size=32, epochs=3, max_length=256